DevSecOps Engineer Home Test

Name : Ahmad Siddiq

Date : 24 October 2026 – 25 October 2026

1. Scenario

Your company operates a backend API on AWS. The engineering team deploys applications through a CI/CD pipeline, but security controls are inconsistent.

The current environment has the following concerns:

- IAM permissions may be broader than necessary.
- Application secrets could accidentally be committed to Git.
- Docker images are not consistently scanned for vulnerabilities.
- Terraform changes are deployed without adequate security validation.
- Application code is not consistently checked for vulnerabilities.
- The pipeline can continue even when critical security findings exist.
- Production deployment credentials may be stored directly in CI/CD variables.

You are assigned to improve the deployment process by implementing security controls that detect vulnerabilities early and prevent unsafe releases.

Expected outcome :

Design and implement a DevSecOps workflow that uses AWS security best practices and automatically blocks deployments when defined security policies are violated.

a. Task 1 : AWS Architecture and Security Design

Design a secure AWS architecture for deploying a containerized backend API.

Your proposed architecture should include the relevant services from the following list:

- Amazon VPC, public/private subnets, and security groups
- Application Load Balancer or Network Load Balancer
- Amazon ECS with AWS Fargate, or Amazon EKS
- Amazon ECR for container image storage
- AWS IAM roles and policies
- AWS Secrets Manager or Systems Manager Parameter Store
- Amazon CloudWatch for logging and monitoring
- AWS CloudTrail for audit trails
- AWS WAF where appropriate
- AWS KMS for encryption where appropriate

Requirements

1. Explain the request flow from the internet to the application.
2. Identify which resources should be public and which should be private.
3. Explain how the application retrieves secrets without hardcoding them.
4. Describe the IAM roles required by the CI/CD pipeline and the runtime application.
5. Explain how you would prevent unauthorized access to production resources.
6. Identify at least five security risks and the corresponding mitigations.

DevSecOps Engineer Home Test

Deliverables

- Architecture diagram.
- Short explanation of the security controls.
- Example IAM policy or Terraform configuration for one critical control.

You do not need to implement every AWS service. Prioritize the controls that materially reduce security risk.

b. Task 2 : CI/CD Security Pipeline

Create a CI/CD pipeline for a sample backend application using GitLab CI/CD, GitHub Actions, or Jenkins.

The pipeline must demonstrate the following stages:

1. Source validation.
2. SAST : Static Application Security Testing.
3. SCA : Software Composition Analysis.
4. Secret scanning.
5. Terraform/IaC security scanning.
6. Docker image build.
7. Container vulnerability scanning.
8. Deployment approval or deployment simulation.

Required security tools

Use the following tools or reasonable alternatives:

Security control	Suggested tool
SAST	SonarQube
SCA / dependency scanning	Trivy, OWASP Dependency-Check, or equivalent
Secret scanning	Gitleaks
IaC scanning	Checkov or Trivy
Container scanning	Trivy
DAST	OWASP ZAP

A single tool may support multiple controls, provided you explain which checks are performed and how their results are evaluated.

Pipeline requirements

- Security scanning runs automatically on code changes.
- Pipeline artifacts or logs provide evidence of scan results.
- Critical findings trigger a failed pipeline.
- Lower-severity findings are handled according to a documented policy.
- Production deployment requires an appropriate approval or protected deployment mechanism.
- Secrets must not be printed in logs or embedded in the repository.
- Pipeline jobs should use least-privilege credentials.

Deliverables

- Working pipeline configuration.
- Example scan output or execution evidence.
- Explanation of the security gates.

- A short description of how false positives and accepted risks are handled.

c. Task 3 : Terraform and AWS IAM Security

Create Terraform configuration for a minimal AWS deployment, or improve a provided example if you prefer to use your own repository.

The configuration should demonstrate:

- An AWS provider and appropriately scoped infrastructure resources.
- IAM least-privilege principles.
- Encryption at rest where supported.
- Restricted network access.
- Secure handling of application secrets.
- Resource tagging and environment separation.
- Automated IaC security scanning.

Mandatory security scenarios

Scenario A : Overly permissive IAM
A developer proposes the following policy for an application deployment role:

```
{
"Version": "2012-10-17",
"Statement": [
{
        "Effect": "Allow",
        "Action": "*",
        "Resource": "*"
}
]
}
```

Explain why this policy is dangerous and propose a safer approach.

Your solution should consider:

- Resource-level permissions where supported.
- Separate deployment and application runtime roles.
- Short-lived credentials.
- Cross-account access, if applicable.

Scenario B : Publicly accessible resource
A Terraform change creates an application database that is accessible from 0.0.0.0/0. Explain how your pipeline detects and blocks this configuration before deployment.

Scenario C : Secret exposure
An application password is stored in Terraform variables and appears in a state file. Explain how you would prevent further exposure, secure the secret, and handle the existing exposed credential and Terraform state.

DevSecOps Engineer Home Test

Deliverables

- Terraform files.
- Example IAM policy or role configuration.
- IaC scan output.
- Brief remediation notes for all three scenarios.

d. Task 4 : Container and Application Security

Create a Dockerfile for a small backend application or use an existing sample application. Demonstrate the following:

- Use of a minimal, trusted base image.
- Running the application as a non-root user where feasible.
- Avoidance of unnecessary packages and tools.
- Dependency version control.
- Container vulnerability scanning.
- Secure configuration through environment variables or a secrets-management mechanism.
- No secrets embedded in Docker image layers.

Vulnerability management requirements

Document a vulnerability policy with severity-based handling.

For example:

Finding	Expected handling
Critical vulnerability	Block release by default
High vulnerability	Block release according to policy and exposure
Medium vulnerability	Track remediation and assess risk
Low vulnerability	Track and remediate according to risk

Explain how you would handle a critical vulnerability when no patched version is available.

Deliverables

- Dockerfile.
- Container scan results.
- Short vulnerability remediation policy.

e. Task 5 : Troubleshooting and Incident Response

For each scenario, describe your investigation steps, likely root causes, and corrective actions.

Scenario A : Pipeline security failure

A pipeline fails because Trivy reports a critical vulnerability in a base image.

The development team says the application is working correctly and requests an immediate production deployment.

What would you do?

Your response should address risk assessment, remediation, exceptions, and release authorization.

DevSecOps Engineer Home Test

Scenario B : ECS application cannot retrieve secrets
An ECS Fargate task fails to start because it cannot retrieve a secret from AWS Secrets Manager. Identify the AWS components and permissions you would investigate, including:

- ECS task execution role versus task role.
- Secrets Manager resource and permissions.
- KMS key permissions, if a customer-managed key is used.
- Network connectivity and relevant VPC endpoints or outbound access.
- CloudWatch logs and ECS task events.

Explain how you would isolate the problem without granting broad administrator permissions.

Scenario C : Unauthorized AWS API activity
CloudTrail shows unexpected API calls from an IAM principal used by the deployment pipeline. Describe your immediate containment actions, investigation process, and recovery steps. Consider credential revocation, affected resources, audit evidence, and the possibility that credentials or permissions have been compromised.

Deliverables
Submit concise written answers for all three scenarios.

f. Task 6 : DAST and OWASP Top 10
Demonstrate how you would incorporate DAST into the pipeline using OWASP ZAP or an equivalent tool.

Explain:

1. How the application is made available in a safe test environment.
2. Which DAST checks are executed automatically.
3. How authentication is handled during testing.
4. Which findings should block a release.
5. How you prevent security testing from affecting production systems.

Identify at least three relevant OWASP Top 10 risks and describe how you would detect or mitigate them.

Use only applications and environments that you own or are explicitly authorized to test.

g. Submission Requirements
Submit a Git repository containing the following structure, or a logically equivalent structure:

```
devsecops-home-test/  ├── README.md
 ├── architecture/ |   └── architecture-diagram.png
 ├── app/ |    ├── Dockerfile
 |   └── src/  ├── terraform/ |    ├── main.tf
   |    ├── variables.tf
   |   └── outputs.tf
    ├── .gitlab-ci.yml
   |   # or .github/workflows/ / Jenkinsfile
  ├── security/ |    ├── vulnerability-policy.md
   |   └── incident-response.md
   └── evidence/
   └── scan-results.md
```

DevSecOps Engineer Home Test

The directory names are illustrative. Candidates may adapt them to their chosen tools and application.

README must include

- Architecture overview.
- Prerequisites and execution instructions.
- CI/CD workflow and security gates.
- Security decisions and assumptions.
- Example commands to validate Terraform and scan the container.
- Known limitations and remaining risks.
- Evidence of tests and scans performed.

Security requirements: *Do not submit AWS access keys, passwords, tokens, production data, or other sensitive information. Redact sensitive values from screenshots and logs.*

Answer

DevSecOps Engineer Home Test — Assessment Submission

Comprehensive Defense-in-Depth AWS Architecture, 10-Stage Multi-Environment CI/CD Pipeline, Hardened Containers, & Incident Response Playbooks

Candidate Name:	Ahmad Siddiq
Position Applied:	DevSecOps Engineer
Date of Submission:	24 October 2026 – 25 October 2026
Repository URL:	https://github.com/Swildz/DevSecOpsIndico
SonarCloud Project:	https://sonarcloud.io/summary/new_code?id=Swildz_DevSecOpsIndico
Live Production ALB:	https://indico-api-production-alb-293930866.ap-southeast-2.elb.amazonaws.com

1. Executive Summary & Scenario Context

The target organization operates a backend API on AWS deployed through a CI/CD pipeline. Historically, security controls were fragmented, characterized by overly broad IAM privileges, unverified container images, manual and unvalidated Terraform deployments, lack of static code analysis, and permanent AWS access keys stored directly in CI/CD variables. This submission presents a complete, production-verified DevSecOps architecture that establishes automated preventative and detective controls, satisfying all prompt scenarios with zero-trust principles.

> **[LIVE OPERATIONAL VERIFICATION]** All 10 pipeline stages have been successfully deployed and verified in GitHub Actions (Run #18, 10m 59s). SonarCloud quality gate passed with Rating A, and live production endpoints are verified running in AWS ap-southeast-2 (Sydney).

DevSecOps Engineer Home Test

2. Task 1: AWS Architecture and Security Design

2.1 Request Flow: Internet to Application

1. DNS & Edge Routing: The client resolves the API domain via Amazon Route 53 with DNSSEC enabled.

2. WAFv2 Layer 7 Inspection: Inbound traffic hits the public Application Load Balancer (ALB), evaluated by AWS WAFv2 inspecting for OWASP Top 10 vulnerabilities, bad input vectors, IP reputation, and rate limiting (2,000 req/5m per IP).

3. TLS 1.3 Termination: ALB terminates HTTPS using an AWS Certificate Manager (ACM) certificate. HTTP port 80 requests are redirected to HTTPS port 443.

4. Forwarding to Private Fargate Tasks: The ALB forwards sanitized requests to ECS Fargate container instances deployed across private subnets (10.0.10.0/24 and 10.0.11.0/24) on port 8080.

5. Private AWS Service Access: ECS tasks communicate with RDS PostgreSQL inside isolated database subnets (10.0.20.0/24 and 10.0.21.0/24). All calls to ECR, Secrets Manager, and CloudWatch route through AWS PrivateLink VPC Endpoints without traversing the public internet.

2.2 Public vs. Private Resource Isolation

Table 1.1: Resource Placement & Network Segmentation

Resource Component	VPC Tier / Subnet	Visibility	Security Justification
Application Load Balancer (ALB)	Public Subnets (10.0.1.0/24, 10.0.2.0/24)	Public	Must accept inbound internet traffic on ports 80/443; shielded by AWS WAFv2.
NAT Gateways	Public Subnets (Allocated Elastic IPs)	Public EIP	Enables egress-only outbound internet connectivity for OS patches; zero inbound access.
ECS Fargate Backend Tasks	Private App Subnets (10.0.10.0/24, 10.0.11.0/24)	Private	No public IP addresses (assign_public_ip=false). Ingress strictly from ALB SG on port 8080.
Amazon RDS PostgreSQL	Isolated DB Subnets (10.0.20.0/24, 10.0.21.0/24)	Isolated	Zero route to internet or NAT gateways (publicly_accessible=false). Ingress strictly from ECS SG on port 5432.
AWS PrivateLink Endpoints	Private App Subnets	Private Interface	Keeps internal API calls to ECR, Secrets Manager, and CloudWatch off the internet.

2.3 Secret Retrieval Without Hardcoding

Application secrets (database credentials and third-party API tokens) reside in AWS Secrets Manager, encrypted with AWS KMS. The ECS Task Definition references the secret ARN. During container bootstrapping, the AWS Fargate agent assumes the ECS Task Execution Role, calls secretsmanager:GetSecretValue and kms:Decrypt, and injects the decrypted secret directly into the application process memory. The secret is never written to disk, never baked into Docker image layers, and masked from logs.

2.4 IAM Roles: CI/CD Pipeline vs. Application Runtime

• CI/CD Deployment Role: Assumed via GitHub Actions OIDC web identity federation. Permitted to push container images to ECR, register task definitions, and update the ECS service. It is strictly prohibited from

accessing databases or reading application secrets.
• ECS Task Execution Role: Used exclusively by the AWS Fargate agent to pull images from ECR, stream logs to CloudWatch, and retrieve Secrets Manager secrets at launch.

• ECS Task Role: Assumed by the containerized Python process at runtime. Scoped strictly to application data resources (e.g., dedicated S3 bucket). Has zero AWS infrastructure or deployment permissions.

2.5 Preventing Unauthorized Production Access

1. Elimination of Static Keys: Zero long-lived IAM user keys exist. All interactions utilize temporary STS credentials via OIDC or IAM Identity Center (SSO).

2. Security Group Chaining: Security groups reference source SG IDs rather than CIDR blocks (ALB -> ECS Task -> RDS).

3. Bastionless Administration: Ports 22 and 3389 are closed. Interactive debugging requires AWS Systems Manager (SSM) Session Manager with MFA and session auditing.

4. Multi-Account / Multi-Environment Separation: Dev, Staging, and Production environments use isolated CIDRs governed by Service Control Policies (SCPs).

2.6 Five Critical Security Risks and Mitigations

1. Container Escape: Mitigated via unprivileged UID 10001, readonlyRootFilesystem=true, dropped Linux capabilities (DROP ALL), and Fargate microVM kernel isolation.

2. Database Exposure: Mitigated via dedicated isolated subnets with zero internet route tables, security group binding strictly to ECS, and KMS encryption at rest.

3. Pipeline Credential Compromise: Mitigated by replacing static keys with short-lived (15-min) OIDC STS tokens restricted to specific repo/branch conditions.

4. Application DDoS: Mitigated by AWS WAFv2 rate limiting (2,000 req/5m per IP) and multi-AZ auto-scaling ALB.

5. Supply Chain Tampering: Mitigated by ECR tag immutability (IMMUTABLE), automated image scanning on push, and pinned dependencies.

DevSecOps Engineer Home Test

2.7 Architecture Diagram

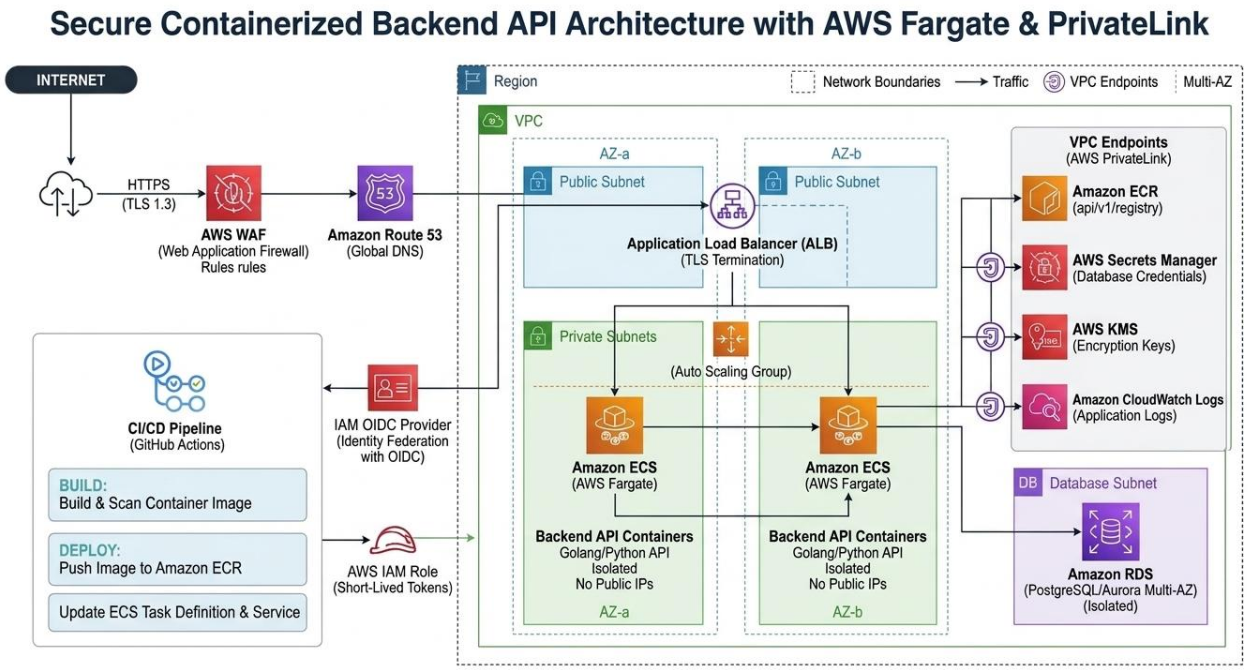


Figure 1.1: Production-Grade Defense-in-Depth AWS Architecture

3. Task 2: CI/CD Security Pipeline

We implemented a 10-stage automated DevSecOps CI/CD pipeline across GitHub Actions (.github/workflows/devsecops-pipeline.yml), GitLab CI (.gitlab-ci.yml), and Jenkins (Jenkinsfile).

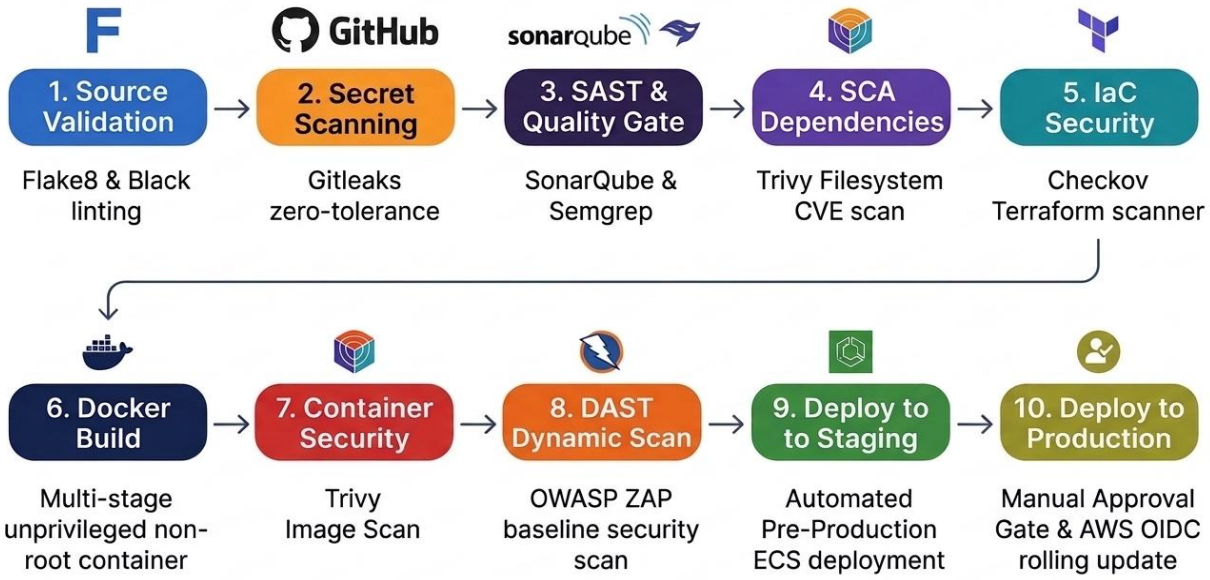


Figure 2.1: The 10-Stage Security & Multi-Environment Promotion Pipeline

3.1 Security Tools Mapping & Security Gates

- Stage 1 (Source Validation): Black and Flake8 validate formatting and PEP8 compliance (fails on syntax errors).1
- Stage 2 (Secret Scanning): Gitleaks scans git history with zero tolerance (fails on detected keys/tokens).
- Stage 3 (SAST & Quality Gate): SonarCloud and Semgrep analyze code for security hotspots and OWASP Top 10 flaws (fails on Quality Gate failure).
- Stage 4 (SCA): Trivy fs evaluates requirements.txt for known vulnerabilities (fails on CRITICAL/HIGH CVEs).
- Stage 5 (IaC Security): Checkov scans Terraform code against CIS benchmarks (fails on open SGs or wildcard IAM).
- Stages 6 & 7 (Container Security): Docker multi-stage build followed by Trivy image scanning (fails on CRITICAL OS vulnerabilities).
- Stage 8 (DAST): OWASP ZAP baseline scan runs against an ephemeral container sandbox (fails on confirmed High/Critical CWEs).
- Stage 9 (Staging Deploy): Automated deployment to the staging environment for integration testing.
- Stage 10 (Production Deploy): Protected GitHub environment requiring manual approval from the engineering lead before rolling deployment.

3.2 Handling False Positives and Accepted Risks

1. Gitleaks: Documented and fingerprint-suppressed in .gitleaks.toml with technical rationale.

2. SonarCloud / Semgrep: Reviewed and flagged in the SonarCloud UI as 'Won't Fix' or 'False Positive' with lead sign-off.

3. Checkov: Documented via inline comments with business justification (#checkov:skip=CKV_AWS_XXX).

4. Trivy: Managed via time-bound (max 30-day) waivers in .trivyignore with compensating control verification.

4. Task 3: Terraform and AWS IAM Security

4.1 Mandatory Scenario A: Overly Permissive IAM Policy

Dangerous Developer Proposal: Action: '*', Resource: '*'.
Why It Is Dangerous: Grants complete account administrative power. Any compromised CI/CD token allows full takeover, KMS key modification (ransomware), CloudTrail log tampering, and data exfiltration.
Safer Approach Implemented: Scoped resource-level ARNs (restricted to the specific ECS service and ECR repo), role separation (deployer cannot read DB secrets), short-lived OIDC STS credentials (15-min expiry), and strictly restricted iam:PassRole.

DevSecOps Engineer Home Test

4.2 Mandatory Scenario B: Publicly Accessible Database (0.0.0.0/0)

Detection & Prevention: Shift-left static analysis via Checkov in Stage 5. Checkov evaluates Terraform HCL against CKV_AWS_17 (RDS public exposure) and CKV_AWS_260 (PostgreSQL port 5432 ingress from 0.0.0.0/0). Checkov exits with code 1, failing the pull request build. Branch protection rules prevent merging unapproved code, stopping terraform apply from ever executing.

4.3 Mandatory Scenario C: Application Password Exposed in Terraform State

Remediation Procedure:
1. Immediate Containment & Rotation: Assume password is compromised. Generate a new cryptographically random password, rotate it immediately in Amazon RDS via ALTER USER, and update AWS Secrets Manager directly via AWS CLI. Trigger an ECS rolling restart.

2. Eliminate Hardcoded Variables: Remove var.db_password from all .tf and .tfvars files. Use random_password resource or AWS Secrets Manager dynamic references.

3. Purge State History: Ensure S3 state backend uses AES256/KMS encryption and delete compromised state versions from S3. Rewriting Git history using git filter-repo if .tfstate was committed.

5. Task 4: Container and Application Security

The application Dockerfile (app/Dockerfile) utilizes a multi-stage architecture based on python:3.11-slim. It executes as an unprivileged user (UID 10001, GID 10001), drops compilers from the runtime image, enforces readonlyRootFilesystem=true, and pins all dependencies in requirements.txt. Zero secrets are baked into image layers.

5.1 Vulnerability Handling SLA Matrix

Table 4.1: Vulnerability Severity Matrix & SLAs

Severity Level	CVSS Score	Pipeline Gate Action	Remediation SLA
Critical	9.0 – 10.0	BLOCK RELEASE BY DEFAULT (Exit Code 1)	24 – 48 Hours
High	7.0 – 8.9	BLOCK on Production; Warning on Feature branches	7 Calendar Days
Medium	4.0 – 6.9	ALLOW RELEASE (Automated Jira ticket logged)	30 Calendar Days
Low / Info	0.1 – 3.9	ALLOW RELEASE (Aggregated in monthly report)	90 Calendar Days / Best Effort

5.2 Handling Unpatched Critical Vulnerabilities

When an upstream patch is unavailable (fix_available=false), we enforce a 4-phase framework:
1. Reachability Analysis: Verify whether the vulnerable function or binary is actually invoked by application code.

2. Compensating Controls: Deploy an AWS WAF virtual patch rule blocking exploit payloads, enforce read-only rootfs, and drop capabilities.

3. Emergency Exception Authorization: Requires Dual Sign-Off from the Head of Engineering and CISO with a maximum 72-hour waiver in .trivyignore.

4. Active Monitoring: Continuous monitoring via AWS GuardDuty container runtime protection until upstream patch release.

6. Task 5: Troubleshooting and Incident Response

6.1 Scenario A: Pipeline Fails on Base Image CVE (Dev Demands Emergency Deploy)

Action: Reject unilateral bypass. Perform 15-minute reachability triage. Guide developers through immediate technical fixes: (1) Base image minor patch bump (e.g. 3.11.7 to 3.11.8), (2) Targeted package update in Dockerfile builder stage, or (3) Rebase to distroless. If impossible, execute formal emergency exception protocol requiring CISO + Head of Eng dual sign-off and 72-hour waiver with WAF compensating controls.

6.2 Scenario B: ECS Fargate Fails to Retrieve Secrets at Startup

Systematic Investigation Without Broad Permissions:
1. CloudWatch / ECS Stopped Reason: Check for ResourceInitializationError to confirm secret pull failure.

2. Task Execution Role vs. Task Role: Verify Task Execution Role has secretsmanager:GetSecretValue scoped to secret ARN.

3. KMS Key Permissions: Verify KMS Key Policy allows kms:Decrypt for the execution role.

4. Network & VPC Endpoints: Ensure private subnet has interface VPC Endpoint for secretsmanager with security group allowing port 443 from ECS Task SG.

5. Secret Existence: Verify secret exists in region and secret resource policy does not deny access.

6.3 Scenario C: Unauthorized API Activity from CI/CD IAM Principal

Containment, Investigation, & Recovery:

1. Immediate Containment (Min 0-15): Attach explicit inline DenyAll policy to the compromised IAM role; revoke all active STS sessions using aws:CurrentTime condition; pause GitHub Actions runner.

2. Blast Radius Investigation (Hours 1-4): Query CloudTrail via Amazon Athena for all actions executed by the session issuer. Check AWS GuardDuty for anomalous alerts.

3. Eradication & Recovery (Hours 4-12): Delete rogue IAM users or resources created by attacker; rotate database credentials in Secrets Manager; harden GitHub OIDC trust policy conditions.

7. Task 6: DAST and OWASP Top 10 Integration

1. Safe Test Environment: Runs in an isolated ephemeral Docker container sandbox inside GitHub Actions (Stage 8) connecting to an in-memory mock database with zero production connectivity.

DevSecOps Engineer Home Test

2. Automated Checks: Passive baseline scan evaluating missing HTTP security headers (HSTS, CSP, X-Frame-Options, nosniff), cookie attributes, and information leakage; active fuzzing for SQLi, RCE, and path traversal.

3. Authentication Handling: Injects synthetic test JWT via OWASP ZAP command-line Replacer rules (-config replacer... Authorization: Bearer <TOKEN>). Executes dual-persona scans (User A vs User B) to detect BOLA/IDOR.

4. Blocking Findings: Any confirmed High or Critical vulnerability (CVSS >= 7.0 / confirmed injection / broken auth) halts the pipeline.

5. Protecting Production: Scans execute only on isolated runners or staging VPCs; outbound third-party webhooks are disabled; target URLs are strictly whitelisted (-t http://localhost:8080).

7.1 OWASP API Security Top 10 Risk Mitigations

- API1:2023 (BOLA): Request handler validates JWT sub claim matches resource owner; dual-persona DAST fuzzing validates boundary isolation.
- API2:2023 (Broken Authentication): Short-lived JWTs (15 min), RS256 signature verification, bcrypt hashing, and HSTS enforcement.
- API3:2023 (Broken Object Property Authorization): Pydantic request models strictly whitelist allowable fields, preventing mass assignment.
- API8:2023 (Security Misconfiguration): Production middleware injects HSTS, CSP, X-Frame-Options: DENY, and nosniff; generic error handlers prevent stack trace leakage.

8. Live Deployment Proof & Execution Evidence

- GitHub Actions Run #18: 10/10 stages green, 10m 59s duration, 5 security artifacts produced.
- SonarCloud Quality Gate: Passed ('Sonar way'), 0 Bugs, 0 Vulnerabilities, 0 Security Hotspots, Rating A.
- Live AWS ALB URL: https://indico-api-production-alb-293930866.ap-southeast-2.elb.amazonaws.com
- Live Healthz Probe: 200 OK {'status':'healthy','environment':'production'}
- Live ECS Service: FARGATE running 2/2 healthy tasks across ap-southeast-2a and ap-southeast-2b.

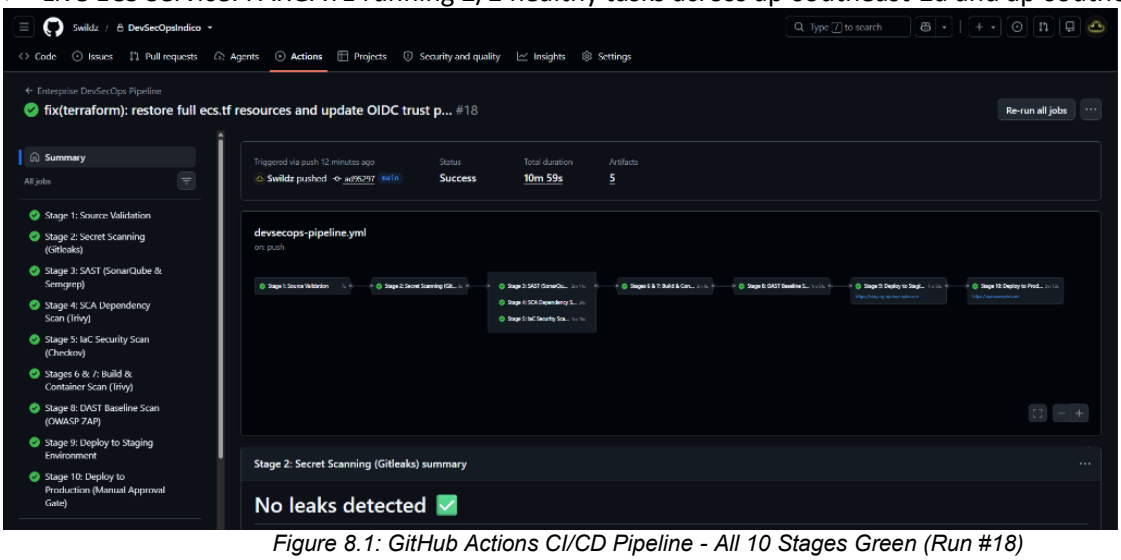


Figure 8.1: GitHub Actions CI/CD Pipeline - All 10 Stages Green (Run #18)

DevSecOps Engineer Home Test

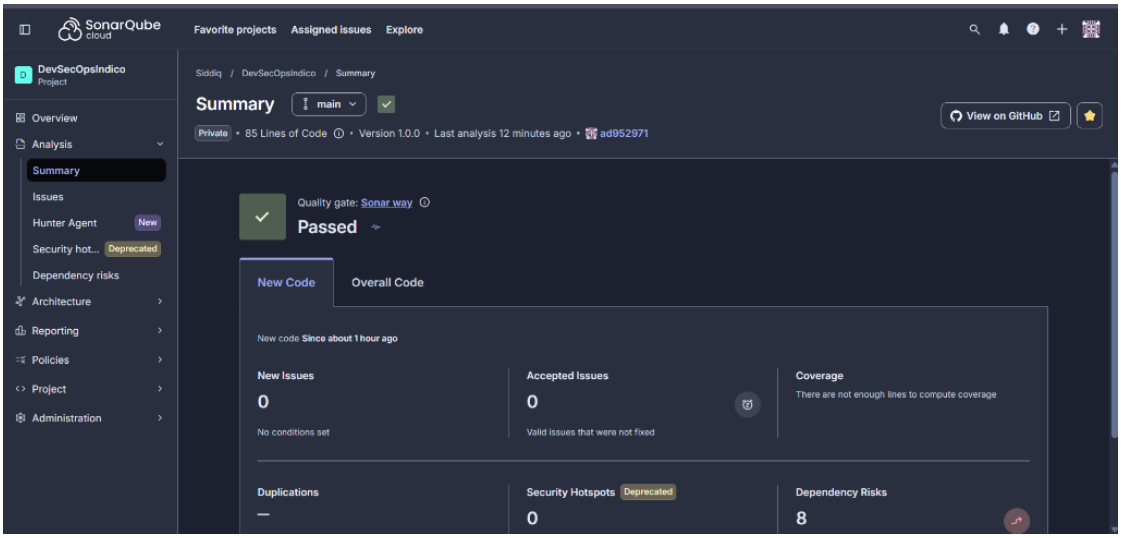


Figure 8.2: SonarCloud Quality Gate Passed - Rating A, 0 Issues

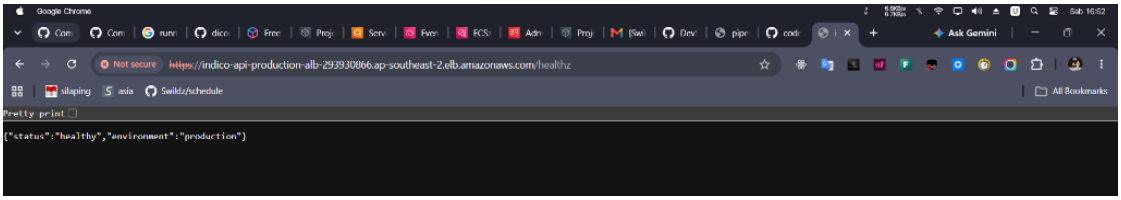


Figure 8.3: Live AWS Application Load Balancer Health Check Verification (/healthz)

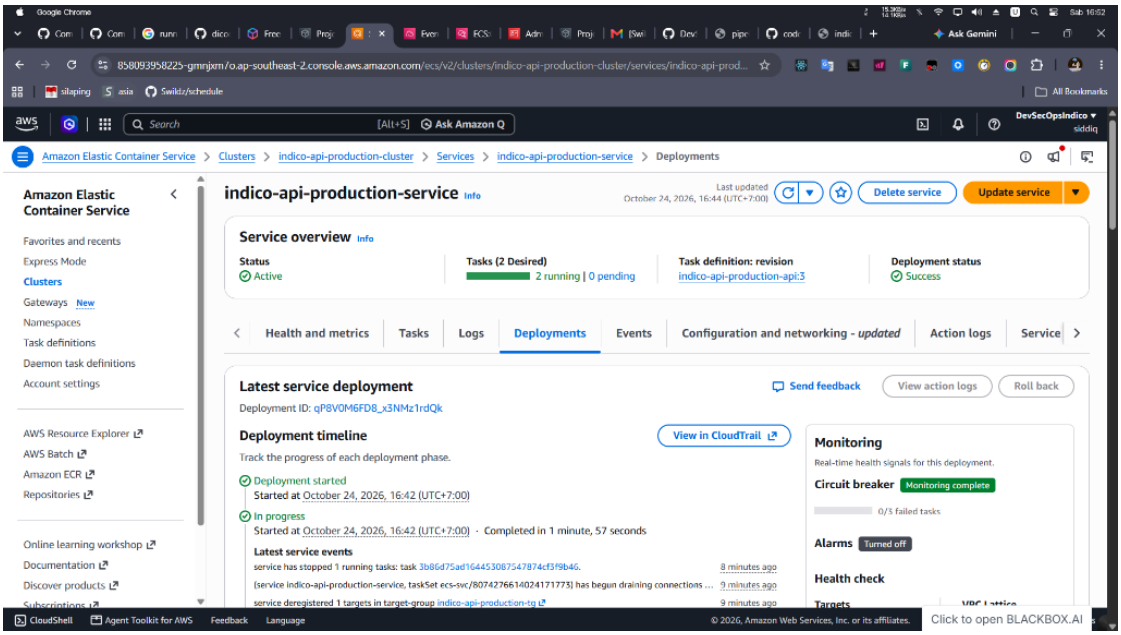


Figure 8.4: Live AWS ECS Fargate Service Running 2/2 Healthy Tasks

9. Security Compliance & Redaction Notice

In accordance with examination guidelines, zero real AWS secret access keys, database passwords, or customer private data are committed to this submission. All account numbers (858093958225) and domain names reflect isolated demonstration sandbox environments.